

Multi-Stage Builds

Build → Test → Runtime · Minimal Final Image

■ Analogy

- Multi-stage builds are like assembling furniture. You need big tools (drill, saw) to build it, but once done, you put away the tools — only the finished furniture goes in your living room. The final Docker image only contains what's needed to run, not the tools used to build it.

Key Terms

Stage	Each FROM instruction in a Dockerfile starts a new stage with its own filesystem
AS alias	Names a build stage — FROM maven:3.9 AS builder — so later stages can reference it
COPY --from	Copies files from a previous stage: COPY --from=builder /app/target/app.jar /app.jar
Build stage	Heavy stage with JDK, Maven, build tools — discarded in final image
Runtime stage	Lean stage with only JRE + artifact. This is what ships to production
Distroless	Google's minimal images with no shell, no package manager — only the runtime
BuildKit	Modern Docker build engine — parallel stages, better caching, secrets support
--target flag	Stop build at a specific named stage: docker build --target tester .
Cache mount	RUN --mount=type=cache,target=/root/.m2 — cache Maven repo across builds
Build secret	RUN --mount=type=secret,id=npm_token — inject secrets without baking into layers

Multi-Stage Dockerfile — Java Spring Boot

```
# ■■ Stage 1: Build ████████████████████████████████████████████████████████  
FROM maven:3.9-eclipse-temurin-17 AS builder  
WORKDIR /build  
  
# Cache dependencies separately from source code  
COPY pom.xml .  
  
RUN --mount=type=cache,target=/root/.m2 \  
    mvn dependency:go-offline -q  
  
COPY src ./src  
  
RUN --mount=type=cache,target=/root/.m2 \  
    mvn clean package -DskipTests -q  
  
# ■■ Stage 2: Test (optional – run in CI) ████████████████████████████████  
FROM builder AS tester  
  
RUN --mount=type=cache,target=/root/.m2 \  
    mvn test  
  
# ■■ Stage 3: Runtime ████████████████████████████████████████████████████████  
FROM gcr.io/distroless/java17-debian12 AS runtime  
COPY --from=builder /build/target/app.jar /app.jar  
EXPOSE 8080  
  
ENTRYPOINT ["java", "-Xmx512m", "-jar", "/app.jar"]
```

Approach	Base Image	Typical Size	Has Shell?
Single stage (JDK)	eclipse-temurin:17-jdk	~600 MB	Yes
Single stage (JRE)	eclipse-temurin:17-jre	~250 MB	Yes
Multi-stage (JRE Alpine)	eclipse-temurin:17-jre-alpine	~120 MB	Yes
Multi-stage (Distroless)	gcr.io/distroless/java17	~85 MB	No

```
# Build only the runtime stage (default)
docker build -t myapp:latest .

# Build only up to 'tester' stage (run tests in CI)
docker build --target tester -t myapp:test .

# Enable BuildKit for cache mounts
DOCKER_BUILDKIT=1 docker build -t myapp:latest .
```

- Netflix uses multi-stage Docker builds for all Java microservices. By separating the Gradle build stage from the JRE runtime stage, they cut image sizes from 900MB to under 150MB per service — saving significant ECR storage and pull time at scale across thousands of containers.